

# 集群软硬件及运维基础

## 目 录

|                                   |    |
|-----------------------------------|----|
| 1 引言：为什么需要集群 .....                | 3  |
| 2 网络协议栈基础 .....                   | 3  |
| 2.1 从两台计算机连接开始 .....              | 3  |
| 2.2 以太网帧与 MAC 地址 .....            | 3  |
| 2.3 IP 地址与子网划分 .....              | 4  |
| 2.4 路由表 .....                     | 4  |
| 2.5 TCP 协议与三次握手 .....             | 5  |
| 2.6 数据包的封装与解封装 .....              | 5  |
| 2.7 非对称加密与安全通信 .....              | 6  |
| 2.8 RDMA：绕过内核的高速通信 .....          | 6  |
| 3 Linux 系统基础 .....                | 7  |
| 3.1 一切皆文件 .....                   | 7  |
| 3.2 用户、用户组与集中认证 .....             | 7  |
| 3.3 文件权限与访问控制 .....               | 7  |
| 3.4 Shell 与 Shell 脚本 .....        | 8  |
| 4 内核与模块管理 .....                   | 8  |
| 4.1 可加载内核模块 .....                 | 8  |
| 4.2 DKMS .....                    | 9  |
| 4.3 eBPF .....                    | 9  |
| 5 集群部署与管理 .....                   | 9  |
| 5.1 PXE 网络启动 .....                | 9  |
| 5.2 NFS Rootfs 与 Overlayfs .....  | 10 |
| 5.3 Spack 包管理器 .....              | 10 |
| 6 HPC 硬件与性能度量 .....               | 10 |
| 6.1 CPU 性能：FLOPS 计算 .....         | 11 |
| 6.2 GPU 性能：以 A100 为例 .....        | 11 |
| 6.3 网络设备：以太网与 InfiniBand .....    | 11 |
| 6.4 NVLink：GPU 间高速互联 .....        | 12 |
| 6.5 IPMI 与 BMC .....              | 12 |
| 7 运维的演进与未来 .....                  | 13 |
| 7.1 算力发展的三个时代 .....               | 13 |
| 7.2 从 Ops 到 DevOps 再到 AIOps ..... | 13 |
| 7.3 全栈运维工程师 .....                 | 13 |
| 8 实战：SSH 免密登录与多机互联 .....          | 14 |

|                |    |
|----------------|----|
| 9 本章你将学会 ..... | 14 |
| 10 要点速查 .....  | 15 |
| 11 小结 .....    | 15 |

# 1 引言：为什么需要集群

当你在自己的笔记本电脑上跑一个程序时，你拥有的算力是有限的：几颗 CPU 核心，几十 GB 内存，一块普通的网卡。对于大规模科学计算，比如天气预报、分子动力学模拟、深度学习训练，这些远远不够。于是人们很自然地想到：把成百上千台机器连接起来协同工作，这不就是一台“超级计算机”吗？

HPC（High-Performance Computing，高性能计算）集群正是这样一种基础设施。它通过高速网络将大量计算节点互联，配合专门的软件栈，使整个集群像一个巨大的计算机一样运行。但要让这么多机器协同工作，我们需要理解一系列基础概念：网络如何连接、Linux 如何管理、内核如何扩展、软件如何部署、硬件如何度量性能、运维如何演进。

本讲将沿这条路径层层递进，从最基础的网络协议栈出发，逐步向上走到 Linux 系统管理、内核模块、集群部署工具，最终触及 HPC 专用硬件、性能度量方法和运维的未来趋势。

## 2 网络协议栈基础

集群中的节点需要频繁通信，网络协议是这一切的基石。我们不妨从一个最简单的问题出发：把两台计算机连起来需要几步？

### 2.1 从两台计算机连接开始

想象你想让两台计算机互相通信。你需要：

1. **第一步**，给两台计算机装上网卡（NIC，Network Interface Card）。网卡是物理层的设备，负责将电信号转换为数字数据。
2. **第二步**，用某种物理介质把两个网卡连起来。这可以是一根网线（双绞线、光纤），也可以是无线信号。
3. **第三步**，操作系统识别并兼容网卡。Linux 内核需要加载对应的驱动程序，才能通过网卡收发数据。

**直觉** 这三步看似简单，却涵盖了 OSI 模型从物理层到应用层的核心思想。真实世界中的网络远比两台直连的计算机复杂：网络中通常有大量设备，需要交换机和路由器来转发数据，需要 IP 地址来跨网寻址，需要 TCP 来保证可靠传输。但万变不离其宗，所有复杂的网络协议都是在这三步的基础上构建的。

### 2.2 以太网帧与 MAC 地址

当网卡被操作系统驱动后，它就可以收发以太网帧（Ethernet Frame）了。以太网是数据链路层（OSI 第二层）最常用的协议。一个以太网帧的结构如下：

| 字段                 | 长度        | 说明                      |
|--------------------|-----------|-------------------------|
| Preamble（前导码）      | 7 B       | 字节序列 0x55，用于接收端时钟同步     |
| SFD（帧起始定界符）        | 1 B       | 0xD5，标记帧的开始             |
| 目的 MAC 地址          | 6 B       | 接收方硬件地址                 |
| 源 MAC 地址           | 6 B       | 发送方硬件地址                 |
| EtherType / Length | 2 B       | 上层协议类型，如 0x0800 表示 IPv4 |
| Payload（负载）        | 46-1500 B | 上层负载数据                  |
| FCS（帧校验序列）         | 4 B       | CRC-32 校验，用于检测传输错误      |

你可能注意到，Preamble 的每一字节都是 0x55（二进制 01010101），这是一个交替的 0/1 模式，目的是让接收端的时钟与发送端同步。紧接的 SFD 是 0xD5（二进制 11010101），它打破了这个规律，告诉接收端“接下来的就是真正的帧数据了”。

以太网帧还支持可选的 **VLAN Tag**（虚拟局域网标签，4 字节），插在 EtherType 之前，用于在同一个物理网络上划分多个逻辑网络。VLAN 在 HPC 集群中常用于隔离计算网络和管理网络。

**直觉** FCS 使用 CRC-32 校验整个帧的内容。如果接收端计算的 CRC 与帧中的 FCS 字段不匹配，说明数据在传输中发生了错误，帧将被丢弃。这是数据链路层保证数据完整性的第一道防线。

**MAC 地址**是 OSI 第二层的硬件地址，用于在同一局域网内唯一标识网络接口。标准的 MAC-48（现称 EUI-48）地址由 48 位（6 字节）组成，典型表示如 00:11:22:33:44:55。其中前 3 字节为 **OUI**（Organizationally Unique Identifier，组织唯一标识符），由 IEEE 分配给设备制造商；后 3 字节由厂商自行管理，以保证全球唯一。

`ip link show`

上述命令会列出本机所有网络接口及其 MAC 地址、MTU 等信息。以太网帧的最小长度为 64 字节（不含 Preamble 和 SFD），最大长度为 1518 字节（不含 VLAN Tag）。当 Payload 不足 46 字节时，MAC 层会自动填充至 46 字节，以满足最小帧长要求，这有助于碰撞检测机制的正常工作。

## 2.3 IP 地址与子网划分

MAC 地址只能在同一局域网内使用。当数据需要跨网传输时，我们需要一个跨网寻址的机制，这就是 **IP 地址**（Internet Protocol Address）。IP 地址是网络层（OSI 第三层）的逻辑地址，用于标识跨越多个局域网的主机。

IPv4 地址为 32 位（4 字节），通常以点分十进制表示，如 192.168.1.10。配合子网掩码（如 255.255.255.0，或 CIDR 记法 /24），IP 地址被划分为网络前缀和主机号两部分。网络前缀标识所属子网，主机号标识子网内的具体设备。

**例** 以 192.168.1.10/24 为例，前 24 位为网络前缀 192.168.1，后 8 位为主机号 10。该子网可容纳  $2^8 - 2 = 254$  台主机，因为全 0 的主机号是网络地址（192.168.1.0），全 1 的主机号是广播地址（192.168.1.255），两者均不可分配给主机。

在 HPC 集群中，计算网络和管理网络通常使用不同的子网，以隔离流量并提升性能。例如，计算节点的计算流量走 10.0.0.0/24 子网，管理流量走 172.16.0.0/24 子网，互不干扰。

IPv6 是 IP 协议的新版本，地址长度为 128 位，解决了 IPv4 地址耗尽的问题。现代 HPC 集群通常同时支持 IPv4 和 IPv6。

## 2.4 路由表

当数据包需要跨子网传输时，路由器根据**路由表**决定下一跳。路由表的核心字段包括：

| 字段            | 说明     | 示例            |
|---------------|--------|---------------|
| Destination   | 目标网络地址 | 0.0.0.0（默认路由） |
| Prefix Length | 前缀长度   | /24           |

|           |              |             |
|-----------|--------------|-------------|
| Next Hop  | 下一跳网关地址      | 192.168.1.1 |
| Interface | 出口网卡         | eth0        |
| Metric    | 路径优先级，值越小越优先 | 100         |

默认路由由 0.0.0.0/0 匹配所有目标地址，指向默认网关。路由器按照“最长前缀匹配”原则选择路由：当多条路由都能匹配目标地址时，选择前缀最长（即最精确）的那条。

**例** 假设路由表中有两条路由：10.0.0.0/8 和 10.1.1.0/24。当目标地址为 10.1.1.5 时，两条路由都能匹配，但 /24 的前缀更长，因此选择 10.1.1.0/24 这条路由。

`ip route show`

上述命令显示当前系统的路由表，你可以看到默认路由、直连子网以及静态配置的路由条目。

## 2.5 TCP 协议与三次握手

TCP（Transmission Control Protocol，传输控制协议）是传输层的可靠连接协议。它通过序列号、确认号和重传机制保证数据的有序性和完整性。TCP 头部包含以下关键字段：

| 字段   | 长度   | 说明                      |
|------|------|-------------------------|
| 源端口  | 16 位 | 发送方端口号                  |
| 目的端口 | 16 位 | 接收方端口号                  |
| 序列号  | 32 位 | 本端数据字节序号                |
| 确认号  | 32 位 | 期望收到的下一个序号              |
| 标志位  | 9 位  | SYN / ACK / FIN / RST 等 |
| 窗口大小 | 16 位 | 流量控制窗口                  |
| 校验和  | 16 位 | 头部和数据校验                 |

TCP 通过“三次握手”（Three-way Handshake）建立连接：

1. **客户端**发送 SYN 报文，序列号设为  $x$ ，表示“我想建立连接”。
2. **服务器**收到后回复 SYN+ACK 报文，序列号设为  $y$ ，确认号设为  $x+1$ ，表示“收到，我也同意”。
3. **客户端**回复 ACK 报文，确认号设为  $y+1$ ，连接建立。

**直觉** 三次握手的核心目标是同步双方的序列号并确认双向通道可用。为什么不两次？因为如果只有两次，服务器无法确认客户端是否收到了自己的 SYN+ACK，可能导致连接不同步。在 HPC 场景中，MPI 进程间的 TCP 连接建立和断开都会产生握手开销，因此高性能互连通常绕过 TCP 直接使用 RDMA。

## 2.6 数据包的封装与解封装

当应用程序发送数据时，数据会经历逐层封装（Encapsulation）。以一个 HTTP 请求为例，数据从上到下依次被封装：

| 层次  | 添加的头部                  |
|-----|------------------------|
| 应用层 | HTTP 请求 GET / HTTP/1.1 |

|     |                              |
|-----|------------------------------|
| 传输层 | TCP 头部：源端口、目的端口、序列号等         |
| 网络层 | IP 头部：源 IP、目的 IP、TTL 等       |
| 链路层 | 以太网帧头：目的 MAC、源 MAC、EtherType |

接收端则反向执行**解封装**（Decapsulation），逐层剥离头部，最终将数据交付给应用程序。以太网帧的最外层是 MAC 地址，IP 包的中间层是 IP 地址，TCP 段的最内层是端口号。这种分层封装的设计使得每一层只需关心自己的功能，极大降低了系统复杂度。

想亲自观察封装过程？可以使用 *wireshark* 抓包工具捕获网络流量，逐层展开查看每一层的头部字段。以访问 <http://clusters.zju.edu.cn/> 为例，你会看到 DNS 解析、TCP 握手、HTTP 请求和响应的完整过程。

## 2.7 非对称加密与安全通信

在集群环境中，节点间的通信安全至关重要。**非对称加密**（Asymmetric Encryption）算法使用一对密钥：公钥和私钥。这种设计带来了两个核心能力：

- **公钥加密，私钥解密**：发送方用接收方的公钥加密数据，只有持有私钥的接收方才能解密，保证**机密性**（Confidentiality）。
- **私钥签名，公钥验证**：发送方用自己的私钥对数据签名，接收方用发送方的公钥验证签名，确认**数据来源和完整性**（Integrity & Authentication）。

**直觉** 想象一个信箱：任何人都能往里面投信（公钥加密），但只有持有信箱钥匙的人才能打开读取（私钥解密）。反过来，如果一个人用私钥“盖章”在信上，任何人都可以用对应的公钥来验证这个章的真伪（签名验证）。

**RSA** 是最经典的非对称加密算法，它保证了两个性质：公私钥无法相互推导，无法从密文推出明文。但 RSA 也有弱点：弱密钥（现在采用 2048 位以上）和低加密指数攻击（如  $e = 3$ ）。

**SSH 免密登录**是非对称加密的典型应用。生成密钥对并将公钥复制到目标节点：

```
ssh-keygen -t ed25519
cat ~/.ssh/id_ed25519.pub >> ~/.ssh/authorized_keys
```

ed25519 是基于椭圆曲线的签名算法，比传统 RSA 更安全、更快速。配置完成后，登录目标节点时无需输入密码，因为 SSH 通过密钥对完成身份认证。在大规模集群中，通常会配合 **LDAP**（Lightweight Directory Access Protocol）或 **Kerberos** 统一管理用户和密钥。

## 2.8 RDMA：绕过内核的高速通信

传统的网络通信路径是：应用程序 → 系统调用 → 内核协议栈 → 网卡。数据需要从用户空间拷贝到内核空间，再由内核发送到网卡，这个过程中 CPU 需要参与数据拷贝和协议处理，延迟较高。

**RDMA**（Remote Direct Memory Access，远程直接内存访问）允许进程直接读写远程内存，无需 CPU 和操作系统介入。数据从一台机器的用户空间内存直接传输到另一台机器的用户空间内存，全程绕过内核。



**直觉** 如果说传统网络通信像寄信（要经过邮局层层中转），RDMA 就像直接把东西递到对方手里，省去了所有中间环节。这就是为什么 InfiniBand 网络（支持 RDMA）能达到微秒级延迟，而普通以太网通常在几十微秒级别。

RDMA 在 MPI 通信中表现优异，是大规模并行计算的首选互连技术。我们将在 HPC 硬件部分进一步讨论 InfiniBand。

## 3 Linux 系统基础

HPC 集群的几乎所有管理操作都在 Linux 系统上进行。Linux 是最广泛的服务器操作系统，由开源社区共建，几乎处处使用。掌握 Linux 基础是运维的前提。

### 3.1 一切皆文件

Linux 的设计哲学之一是“一切皆文件”（Everything is a file）。普通文件、目录、字符设备、块设备、管道（pipe）、套接字（socket）、符号链接（symbolic link）等，均以文件方式抽象。这种统一的接口简化了系统设计，使得 read/write 等系统调用可以通用于各种资源。

**例** 例如，/dev/null 是一个特殊设备文件，写入其中的数据会被丢弃；/proc/cpuinfo 是一个虚拟文件，读取它可以获取 CPU 信息；/dev/stdin 和 /dev/stdout 分别代表标准输入和标准输出。这种设计让你可以用统一的工具（如 cat、grep）操作各种资源，而不需要为每种资源类型学习不同的工具。

### 3.2 用户、用户组与集中认证

Linux 是多用户操作系统，每个用户有自己的身份标识（UID）和所属用户组（GID）。用户组用于将具有相同权限需求的用户归为一类，简化权限管理。

在 HPC 集群中，用户数量可能达到数百甚至数千人，而且需要在所有节点上保持一致。如果每个节点都单独维护一份用户列表，管理成本极高且容易出错。因此，集群通常使用**集中式用户认证**，最常见的是 LDAP（Lightweight Directory Access Protocol，轻量级目录访问协议）。

**直觉** LDAP 就像集群的“户籍管理处”。所有用户信息（用户名、UID、密码、所属组）集中存储在一台 LDAP 服务器上，每个计算节点在用户登录时向 LDAP 服务器查询并验证身份。这样，管理员只需在 LDAP 服务器上添加或删除一个用户，所有节点立即生效。

### 3.3 文件权限与访问控制

Linux 文件权限分为读（r）、写（w）、执行（x）三类，分别针对文件所有者（owner）、所属组（group）和其他用户（others）。使用 ls -l 可以查看权限位。

除了基本的 rwx 权限，Linux 还提供更细粒度的访问控制：

| 工具                | 说明                             |
|-------------------|--------------------------------|
| chattr / lsattr   | 设置文件属性，如不可变属性 +i，即使 root 也无法修改 |
| setfacl / getfacl | ACL（访问控制列表），允许为特定用户或组设置独立权限    |
| sudo / su         | sudo 以管理员身份执行单条命令，su 切换到其他用户   |

在 HPC 集群中，共享存储上的文件权限管理尤为重要。多个用户可能需要读取同一份数据但只有特定用户能修改，ACL 提供了比传统 `rwX` 更灵活的权限控制。

### 3.4 Shell 与 Shell 脚本

**Shell**（外壳）是用户和操作系统间的一个交互接口，用于解释并执行用户输入的命令。常见的 Shell 程序包括 **Bash**（Bourne Again SHell）、**Zsh**（Z Shell）和 **Fish**（Friendly Interactive SHell）。

Shell 脚本以 **shebang** 行开头（`#!/`），指定解释器。以下是一个批量重命名文件的示例：

```
#!/bin/bash
for file in *; do
    mv "$file" "data-$file"
done
```

逐行解释上述脚本：

- `#!/bin/bash`：shebang 行，告诉系统用 `/bin/bash` 来执行这个脚本。
- `for file in *; do`：遍历当前目录下的所有文件，每次循环将文件名赋给变量 `file`。
- `mv "$file" "data-$file"`：将文件重命名，添加 `data-` 前缀。`"$file"` 中的引号防止文件名包含空格时出错。
- `done`：结束循环。

在集群运维中，Shell 脚本常用于批量执行命令、收集日志和自动化部署。掌握 `$()` 命令替换和 `|` 管道符是编写高效脚本的基础。

## 4 内核与模块管理

**Linux 内核**（Linux Kernel）采用宏内核架构，但通过可加载内核模块机制实现了灵活的扩展。

### 4.1 可加载内核模块

为什么需要可加载内核模块？如果所有驱动和功能都编译进内核，内核会变得非常庞大，而且每次添加新功能都需要重新编译和重启。**LKM**（Loadable Kernel Module，可加载内核模块）解决了这个问题：它是 `.ko` 文件，可以在运行时动态加载或卸载，无需重启系统。

**直觉** 内核模块就像“即插即用”的插件。你可以在不关机的情况下把一个新的功能“插入”内核，用完之后再“拔掉”。在 HPC 环境中，InfiniBand 驱动、Lustre 客户端等通常以内核模块形式加载。

内核模块还支持 **Hotplug**（热插拔）机制，当硬件设备被插入或拔出时，内核会自动加载或卸载对应的模块。

常用命令：

```
modprobe nfs
insmod my_driver.ko
lsmod
rmmod nfs
```



- `modprobe nfs`: 自动处理模块依赖关系，加载 `nfs` 模块及其所有依赖。
- `insmod my_driver.ko`: 直接加载指定的 `.ko` 文件，但不处理依赖。
- `lsmod`: 列出当前已加载的所有内核模块。
- `rmmod nfs`: 卸载 `nfs` 模块。

## 4.2 DKMS

当内核升级时，已编译的模块可能因内核接口变化而失效（典型的例子是 NVIDIA 驱动在新内核更新后经常无法工作）。DKMS（Dynamic Kernel Module Support，动态内核模块支持）解决了这一问题。

**直觉** DKMS 的做法是：在安装模块时，不仅安装编译好的 `.ko` 文件，还保存模块的源码到 `source tree` 中。当内核更新时，DKMS 会自动检测到内核版本变化，重新编译模块，确保兼容性。这样，管理员更新内核后无需手动重新编译每个模块。

这对 HPC 集群尤为重要，因为集群操作系统经常需要更新内核以修复安全漏洞或获得新功能。

## 4.3 eBPF

eBPF（extended Berkeley Packet Filter）允许在内核中运行沙箱程序，无需修改内核源码或加载内核模块。eBPF 程序经过验证器检查后安全执行，广泛应用于网络监控、性能分析和安全过滤等场景。

**直觉** 如果说 LKM 是给内核“装插件”，eBPF 更像是给内核“装探针”。你可以在内核的各种 hook 点（系统调用、网络包、内核跟踪点）插入自定义的小程序，收集你关心的数据，而不会影响系统的安全性和稳定性。

eBPF 为 HPC 集群的运维提供了强大的可观性工具，例如用 `bpftool` 追踪系统调用的延迟分布，或者用 eBPF 程序监控网络流量，实现高性能的数据包过滤和负载均衡。

# 5 集群部署与管理

HPC 集群通常包含数十到上千个节点。手动逐台部署和管理是不可行的，需要自动化工具。当你面对几百台机器时，“Too many machines!!!!”不再是一句感叹，而是一个需要工程化解决的挑战。

## 5.1 PXE 网络启动

PXE（Preboot Execution Environment，预启动执行环境）允许计算机通过网络启动，而不依赖本地硬盘。这使得集群可以批量部署操作系统，实现“无盘启动”。

PXE 的工作流程为：

1. 节点网卡固件发送 DHCP 请求。
2. DHCP 服务器返回 IP 地址和引导文件名。
3. 节点通过 TFTP 下载引导程序。
4. 引导程序加载操作系统镜像并启动。

**直觉** 整个过程完全自动化，适合大规模集群的初始部署和系统恢复。想象几百台裸机同时上电，它们不需要有人挨个插 U 盘装系统，而是自动从网络获取引导程序和操作系统镜像。

## 5.2 NFS Rootfs 与 Overlayfs

在无盘节点方案中，节点的根文件系统通过 **NFS** (Network File System) 从共享存储挂载。这样做的好处是“One Place, All kinds of distros, Live Updates”：管理员只需维护一份操作系统镜像，所有节点共享，更新时实时生效。

由于多个节点共享同一份只读根文件系统，需要 **Overlayfs** 提供可写层：

- **lower layer** (下层)：共享的只读 NFS 根文件系统。
- **upper layer** (上层)：每个节点本地的可写层（通常基于 tmpfs）。

**直觉** Overlayfs 实现了“写时复制” (Copy-on-Write)。当节点修改文件时，Overlayfs 将修改写入 upper layer，而不影响 lower layer 的原始文件。节点重启后，upper layer 清空，系统恢复初始状态。这种“无状态”设计大大简化了集群维护：管理员只需更新一份 NFS 镜像，所有节点重启后即可获得新环境。

## 5.3 Spack 包管理器

HPC 环境中经常需要同一软件的多个版本或不同编译选项共存。例如，某些应用需要 OpenMPI 4.1，另一些需要 5.0；某些应用需要 GCC 11，另一些需要 Intel 编译器。传统的系统包管理器难以满足这种需求。

**Spack** 是专为 HPC 设计的包管理器，支持多版本并存和复杂的依赖管理：

```
git clone -c feature.manyFiles=true --depth=2 https://git.zju.edu.cn/zjusct/spack ~/spack
spack install intel-oneapi-vtune
spack load intel-oneapi-vtune
```

- `git clone ...`：克隆 Spack 仓库到 ~/spack 目录。--depth=2 减少下载量。
- `spack install intel-oneapi-vtune`：安装 Intel VTune Profiler，Spack 自动处理编译器选择和依赖关系。
- `spack load intel-oneapi-vtune`：将 VTune 加入当前环境变量，类似于 `module load`。

如果在 VS Code 等工具的 `bash` 终端中 `Spack` 不工作，可能是因为环境变量未正确加载。尝试执行 `echo ". /etc/profile.d/z00_spack.sh" >> .bashrc` 将 `Spack` 初始化脚本加入 `bash` 配置。如果安装出现严重问题，可以执行 `rm -rf ~/spack ~/.spack` 后重新安装。

## 6 HPC 硬件与性能度量

理解 HPC 硬件性能是评估集群能力和优化程序的基础。我们从 CPU、GPU、网络设备和带外管理四个方面展开。

一个典型的 HPC 集群包含以下组件：

| 类别           | 内容                        |
|--------------|---------------------------|
| Node（节点）     | 登录管理节点、CPU 计算节点、GPU 计算节点  |
| Network（网络）  | IPMI 管理网络、计算网络、存储网络       |
| Storage（存储）  | 本地文件系统、网络文件系统（NFS）、并行文件系统 |
| Software（软件） | OS、编译器与运行时、驱动、并行文件系统版本    |

## 6.1 CPU 性能：FLOPS 计算

**FLOPS**（Floating-point Operations Per Second，每秒浮点运算次数）是衡量计算节点性能的核心指标。单节点的理论峰值计算公式为：

$$\text{GFlops} = f \times F \times N_C \times N_{\text{CPU}} \quad (6.1)$$

其中  $f$  为 CPU 主频（GHz）， $F$  为每核每周期浮点运算次数， $N_C$  为核心数， $N_{\text{CPU}}$  为 CPU 数量。

**例** 以 Intel Xeon Platinum 8358 为例：主频 2.6 GHz，32 核，支持 **AVX-512**。AVX-512 配备两个 FMA（Fused Multiply-Add）单元，每个 512 位寄存器可容纳 8 个双精度浮点数，乘加算两次运算，因此每周期每核可执行：

$$8 \times 2 \times 2 = 32 \text{ DP FLOPS/cycle/core} \quad (6.2)$$

理论峰值性能为：

$$2.6 \times 32 \times 32 = 2662.4 \text{ GFlops} \approx 2.66 \text{ TFlops} \quad (6.3)$$

需要注意的是，这是理论峰值，实际应用通常只能达到峰值的 50%-80%，取决于算法的向量化程度和内存带宽。

## 6.2 GPU 性能：以 A100 为例

GPU 在大规模并行计算中的浮点性能远超 CPU。以 **NVIDIA A100 80GB PCIe** 为例，其 FP64 Tensor Core 性能高达 19.5 TFLOPS，远大于 CPU（Platinum 8358 约 2.66 TFlops）的 FLOPS。

GPU 的浮点性能计算方式与 CPU 有所不同，主要取决于 **Tensor Core**（张量核心）的矩阵运算能力。Tensor Core 专门为深度学习的矩阵乘加运算设计，可以在一个时钟周期内完成一个矩阵乘法和一个矩阵加法。

**直觉** CPU 和 GPU 的设计哲学不同：CPU 优化的是单线程延迟，追求在尽可能短的时钟周期内完成复杂任务；GPU 优化的是吞吐量，通过成千上万个简单核心并行工作来获得极高的总计算量。这就是为什么 GPU 在深度学习训练和大规模科学计算中表现突出的原因。

## 6.3 网络设备：以太网与 InfiniBand

HPC 集群的网络设备直接决定了节点间通信的性能。常见的网络设备包括：

| 网络类型         | 带宽        | 说明                      |
|--------------|-----------|-------------------------|
| GbE（千兆以太网）   | 1 Gb/s    | 千兆电口交换机 + 千兆电口网卡，用于管理网络 |
| 10GbE（万兆以太网） | 10 Gb/s   | 万兆电口交换机 + 万兆电口网卡，用于存储网络 |
| InfiniBand   | 100 Gb/s+ | 极高的吞吐量和极低的微秒级延迟，用于计算网络  |

**InfiniBand** 是 HPC 专用的高速互连网络，提供 100 Gb/s 甚至更高的吞吐量和极低的微秒级延迟。相比普通以太网，InfiniBand 采用专用硬件和精简协议栈，支持 RDMA，允许进程直接读写远程内存，无需 CPU 和操作系统介入。

**直觉** 如果说以太网是“公用公路”，所有车辆共享道路、需要遵守红绿灯（TCP 协议栈），那么 InfiniBand 就是“专用高速公路”，数据直达目的地，没有中间站。在 MPI 通信中，InfiniBand 表现优异，是大规模并行计算的首选互连技术。

## 6.4 NVLink: GPU 间高速互联

当一台服务器内有多块 GPU 时，GPU 之间的数据传输需要经过 PCIe 总线，带宽有限。**NVLink** 是 NVIDIA 开发的 GPU 间高速互联技术，提供远高于 PCIe 的带宽，使多 GPU 之间的数据传输更加高效。

以 **NVIDIA DGX A100** 为例，其内部的多块 A100 GPU 通过 NVLink 全互联，支持 GPU 间直接的高速数据交换，无需经过 CPU 中转。这对于多 GPU 并行训练和大规模张量并行至关重要。

## 6.5 IPMI 与 BMC

**IPMI**（Intelligent Platform Management Interface，智能平台管理接口）通过 **BMC**（Baseboard Management Controller，基板管理控制器）实现对服务器的**带外管理**（Out-of-band Management）。

BMC 独立于 BIOS 和操作系统运行，只要接通电源即可工作。它相当于整个平台管理的“大脑”，可以监控各个传感器的数据并记录各种事件日志。BMC 的主要功能包括：

- 监控服务器的温度、风扇速度、电压等硬件参数。
- 记录硬件错误日志。
- 提供“远程 KVM”（键盘、视频和鼠标）功能，允许管理员远程查看和操作服务器的显示输出。
- 通过网络接口远程控制服务器电源，包括开机、关机和重启。
- 在服务器启动过程中修改 BIOS 设置。

```
ipmitool -I lanplus -H <BMC_IP> -U <BMC_USER> -P <BMC_PASSWORD> mc info
```

上述命令通过 IPMI 协议连接到指定 BMC，获取管理控制器的信息。各厂商有自己的 BMC 实现：

| 厂商         | BMC 实现 |
|------------|--------|
| Inspur（浪潮） | iBMC   |
| Dell       | iDRAC  |
| HPE        | iLO    |

|     |     |
|-----|-----|
| IBM | IMM |
|-----|-----|

这些实现均兼容 IPMI 标准。在集群运维中，BMC 是远程管理和故障诊断的关键工具。当节点操作系统无响应时，管理员可以通过 BMC 远程查看控制台、强制重启或重装系统。

## 7 运维的演进与未来

理解 HPC 集群的软硬件只是起点。随着算力需求的不断增长，运维（Operations）这个角色本身也在发生深刻的变化。

### 7.1 算力发展的三个时代

纵观计算技术的发展，我们可以将其划分为三个时代：

| 阶段   | Past（过去）  | Now（现在） | Future（未来）   |
|------|-----------|---------|--------------|
| 时代   | 云计算时代     | 通信时代    | 智算时代         |
| 核心能力 | 基础算力      | 通信网络    | 超级计算机 / 数据中心 |
| 关键词  | 互联化 / 平台化 | 信息化     | 智能化 / 定制化    |

从云计算时代的基础算力，到通信时代的高速互联网络，再到智算时代的模型、数据与 AI 算力，算力基础设施的规模和复杂度不断攀升，对运维提出了越来越高的要求。

### 7.2 从 Ops 到 DevOps 再到 AIOps

运维的角色同样经历了三个阶段的演进：

| 阶段     | 名称     | 核心理念                                    |
|--------|--------|-----------------------------------------|
| Past   | Ops    | 保障各类设备、系统、网络正常运行和可用                     |
| Now    | DevOps | 在应用程序的整个生命周期中工作，发展多项技能                  |
| Future | AIOps  | 利用人工智能自动化关键的 IT 运维任务，提供积极主动、个性化和实时的运维洞察 |

**直觉** 传统的 Ops 关注“保稳定”：设备不坏、系统不宕就行。DevOps 打破了开发和运维的边界，要求工程师具备全栈能力，从开发到部署到运维全程参与。而 AIOps 则是未来的方向：用 AI 识别和预测系统状态，减少人工成本，实现从“被动响应”到“主动预防”的转变。

### 7.3 全栈运维工程师

在 SCT（浙江大学超算团队）等顶尖 HPC 团队中，运维工程师需要成为“全栈工程师”（Full-stack Engineer），既“文能调代码”，又“武能进机房”：

- **硬件层面**：了解底层原理，理解系统各组件间的交互协作。日常维护和基础设施改造。
- **软件层面**：熟悉并行编程范式，指导性能分析优化。集群架构设计与优化，性能分析与功耗控制。
- **后端**：数据采集与清洗，作业调度。
- **前端**：运维可视化，智能分析与交互。



- **安全**：保护网络 and 系统免受恶意攻击和数据泄露。
- **AI**：AI 识别和预测系统状态，减少人工成本。

**直觉** 一句话概括全栈运维的精髓：从大量重复的人肉操作中解放出来，专注于运维服务质量的提升；加速开发、测试和分析流程，专注于程序开发和性能优化。自动化运维建设是当前的重点方向。

## 8 实战：SSH 免密登录与多机互联

现在集群上开放了 3 个节点，让我们配置这三个节点之间的 SSH 互联：

- 节点 sct101
- 节点 m600
- 节点 m601

在每台节点上执行以下步骤：

```
ssh-keygen -t ed25519
cat ~/.ssh/id_ed25519.pub >> ~/.ssh/authorized_keys
```

逐行解释：

- `ssh-keygen -t ed25519`：生成一对 ed25519 密钥，私钥保存在 `~/.ssh/id_ed25519`，公钥保存在 `~/.ssh/id_ed25519.pub`。过程中会提示设置密码短语（passphrase），可以直接回车跳过。
- `cat ~/.ssh/id_ed25519.pub >> ~/.ssh/authorized_keys`：将公钥追加到 `authorized_keys` 文件。SSH 登录时，服务器会检查请求方的公钥是否在 `authorized_keys` 中，如果在则允许免密登录。

要实现三节点互联，需要将每个节点的公钥复制到另外两个节点的 `authorized_keys` 中。可以使用 `ssh-copy-id` 命令简化操作：

```
ssh-copy-id -i ~/.ssh/id_ed25519.pub user@sct101
ssh-copy-id -i ~/.ssh/id_ed25519.pub user@m600
ssh-copy-id -i ~/.ssh/id_ed25519.pub user@m601
```

配置完成后，在三台节点之间执行 ssh 登录将无需输入密码，为后续的 MPI 并行作业部署打下基础。

## 9 本章你将学会

1. 理解以太网帧结构、MAC 地址、IP 地址和子网划分，能够使用 `ip` 命令查看网络配置。
2. 掌握 TCP 三次握手原理和数据包封装解封装过程，理解 RDMA 相比传统网络通信的优势。
3. 能够使用 Linux 基本命令管理文件权限和访问控制，编写简单的 Shell 脚本自动化任务。
4. 理解内核模块（LKM）、DKMS 和 eBPF 的作用，能够使用 `modprobe`、`lsmod` 等命令管理模块。
5. 掌握 PXE、NFS Rootfs、Overlayfs 和 Spack 等集群部署工具的原理和基本用法。
6. 能够计算 CPU 和 GPU 的理论峰值 FLOPS，理解 InfiniBand、NVLink 等高速互联技术的优势。



7. 了解 IPMI/BMC 带外管理的原理，能够使用 ipmitool 远程管理服务器。
8. 理解运维从 Ops 到 DevOps 到 AIOps 的演进趋势，明确全栈运维工程师的技能要求。
9. 能够在集群节点间配置 SSH 免密登录，实现多机互联。

## 10 要点速查

| 主题      | 命令/工具                      | 用途           |
|---------|----------------------------|--------------|
| 网络接口    | ip link show               | 查看网卡和 MAC 地址 |
| 路由表     | ip route show              | 查看路由表和默认网关   |
| SSH 密钥  | ssh-keygen -t ed25519      | 生成密钥对        |
| SSH 免密  | ssh-copy-id                | 复制公钥到远程节点    |
| 文件属性    | chattr / lsattr            | 设置/查看文件属性    |
| 访问控制    | setfacl / getfacl          | 设置/查看 ACL    |
| 内核模块    | modprobe / lsmod           | 加载/查看内核模块    |
| HPC 包管理 | spack install / spack load | 安装/加载软件      |
| 带外管理    | ipmitool                   | 远程管理服务器 BMC  |
| 抓包分析    | wireshark                  | 捕获和分析网络流量    |

## 11 小结

本讲我们从最基础的网络协议出发，理解了数据是如何从应用层逐层封装、通过 MAC 地址和 IP 地址寻址、经由 TCP 保证可靠传输的。在此基础上，我们学习了 Linux 系统的文件抽象、权限管理和 Shell 脚本编程，这些是集群运维的基本功。然后我们深入内核层面，了解了 LKM、DKMS 和 eBPF 如何在不重启系统的前提下扩展内核功能。

在集群层面，我们掌握了 PXE 网络启动实现无盘部署、NFS Rootfs 和 Overlayfs 实现无状态节点、Spack 实现 HPC 软件多版本共存。在硬件层面，我们学会了用 FLOPS 度量 CPU 和 GPU 性能，理解了 InfiniBand 和 NVLink 等高速互联技术如何突破通信瓶颈，以及 IPMI/BMC 如何实现带外远程管理。最后，我们展望了运维从 Ops 到 DevOps 再到 AIOps 的演进趋势，理解了全栈运维工程师的技能要求。

掌握了这些基础知识后，你就有能力理解一个 HPC 集群是如何从硬件到软件、从单机到多机协同工作的。下一讲我们将进入并行编程的世界，学习如何利用这些集群资源编写高效的并行程序。